



AI Engineering Code Summit 2025-11-20: Major Themes & Insights

The 2025 AI Engineering Code Summit brought together leaders from Anthropic, OpenAI, Google Labs, Replit, Bloomberg, Capital One, Stanford, McKinsey, and dozens of other organizations at the forefront of AI-assisted software development. Over the course of a single day, 20+ speakers revealed converging insights about what actually works—and what doesn't—when deploying AI coding agents at scale.

From Stanford's 120,000-developer study to Northwestern Mutual's Fortune 100 implementation challenges, from Zapier's support teams shipping code to Tenex's million-dollar engineers, the conference painted a remarkably consistent picture: AI isn't replacing software engineering fundamentals, it's amplifying them. The organizations succeeding aren't those racing to adopt every new model, but those methodically building foundations—clean code, clear processes, stakeholder trust—that allow AI tools to deliver measurable value.

Six major themes emerged across the day's sessions:

Theme 1: Trust, Role Changes, and What's Good for Humans is Good for AI

When Max Kanat-Alexander, Capital One's Executive Distinguished Engineer for Developer Experience, told a room full of enterprise engineers that "what's good for humans is good for AI," the applause was immediate. It wasn't just a catchy line—it was the crystallization of a fundamental shift happening across organizations grappling with AI adoption. The promise isn't about replacing developers with smarter machines. It's about recognizing that the same infrastructure investments that make human developers productive—better tooling, clearer documentation, robust testing—are exactly what AI agents need to thrive.

This insight arrived at a crucial moment. As Gene Kim observed during his morning conversation with Steve Yegge, "trust is to what degree can I predict how another part will act." And right now, enterprises are discovering that AI trust increases not through hype cycles or vendor promises, but through measurable experience with systems that behave

predictably. The Capital One engineer's message resonated because it reframed the entire AI conversation: stop building special accommodations for agents, and start fixing the foundational problems that have plagued human developers for decades.

Code Review Quality

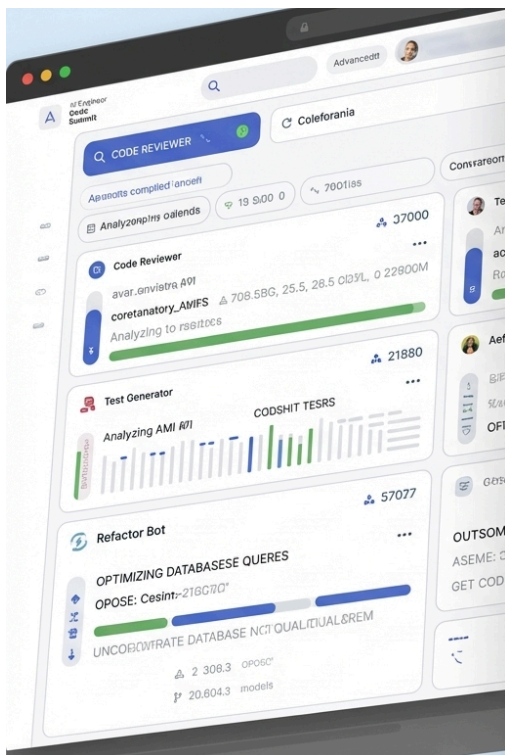
- **You have to encourage great reviews**
- **That hold a high bar and reject code that doesn't meet it**
- **Where junior engineers learn about coding *and* reviewing by getting reviews from seniors.**

The evidence for this approach is already appearing in organizational transformation stories that would have seemed impossible just months ago. Lisa Orr from Zapier dropped what might be the conference's most striking datapoint: support team members using AI coding tools have achieved such proficiency that "support team members who are part of this experiment are moving into the engineering team." These aren't computer science graduates finally getting their shot—these are customer support specialists who understand user pain intimately, now empowered to ship fixes directly. Their secret weapon, Scout (built on Cursor and MCP protocols), helps them diagnose issues, generate fixes, and validate solutions with roughly 70% accuracy. The result? Zapier's support team doubled their velocity, pushing 3-4 fixes per week with Scout's assistance.

This role fluidity extends far beyond support teams. Dan Shipper's Every—a 15-person company running six business units and four software products with seven-figure revenue—operates on a principle that would terrify traditional engineering managers: "Managers can commit code." Not just review it, not just approve it—actually commit it. Each of Every's applications is built by a single developer working with AI agents in parallel, treating code as cheap enough to experiment with multiple approaches simultaneously. New hires

become productive on their first day. Developers casually contribute to products outside their primary ownership. The entire operation runs on what Shipper calls "compounding engineering," where each feature makes the next easier rather than adding to technical debt.

Yet this democratization of code-writing capabilities is exposing uncomfortable truths about organizational psychology. Gene Kim shared war stories from his enterprise tech leadership summit: when Capital One and later Fidelity pushed senior developers to adopt DevOps practices, "senior devs didn't want to do it." Now Cisco requires every senior developer to "vibe code an application to production"—and the resistance patterns are repeating. Meanwhile, Kim noted, completely new practitioners are thriving. The dividing line isn't skill level; it's adaptability and willingness to work differently.



Adoption Depends on Having UI

We've established that CLI agents are too hard for most devs.

We clearly need a UI.

We also need to lift people up.

- Claude Code shows your nose in everything.
- You have to watch everything it does.
- We need helpers -- models!

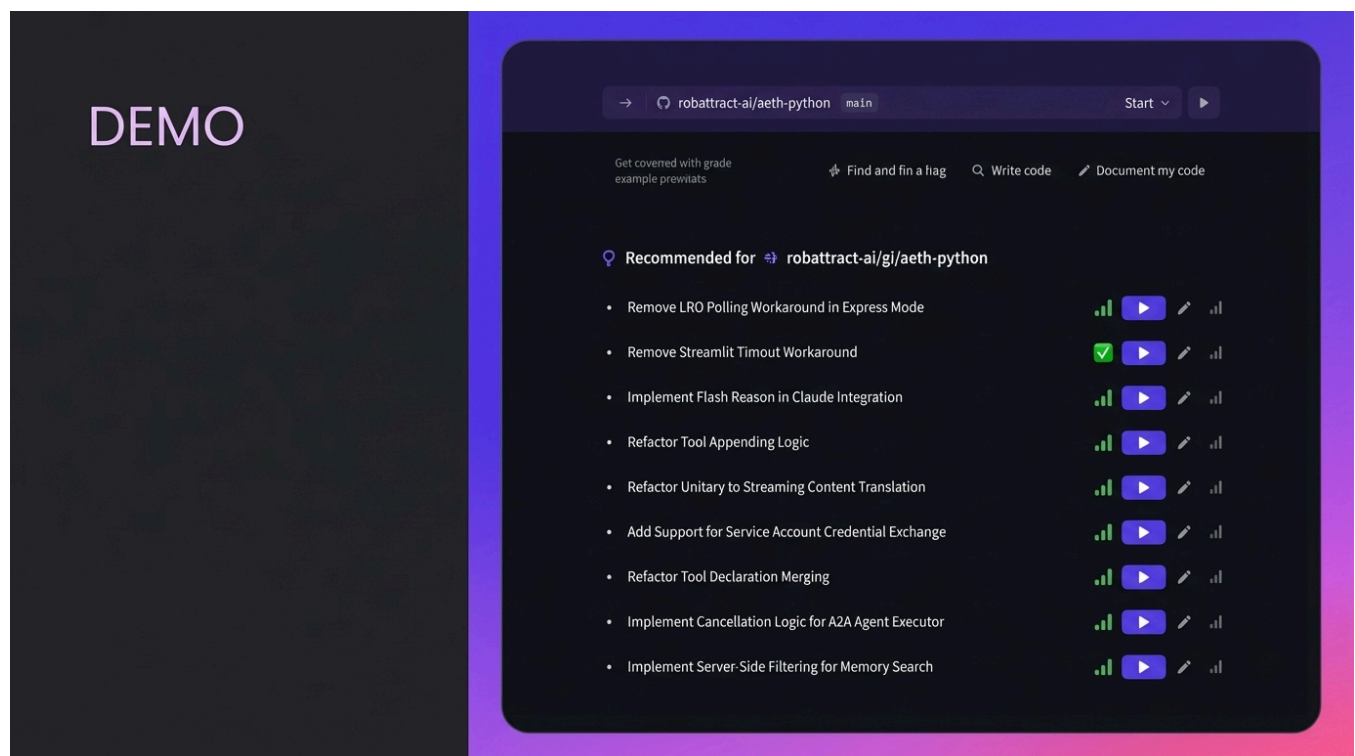
But coding agents aren't heading in this direction. Instead, they're doubling down on the CLI.

- How much has Claude Code changed since February?

Justin Reock from DX framed this challenge through the lens of Google's Project Aristotle research, which found that "the overwhelming predictor [of team success] was psychological safety." His guidance for leadership: "Treat software development as a systems problem, not a people's problem." This reframing is crucial. When organizations approach AI adoption as a trust-building exercise in predictable systems rather than a competency test for individual developers, resistance drops and experimentation increases. DX's AI Measurement Framework tracks utilization, impact, and cost—but the real breakthrough comes from establishing feedback loops that help teams understand *why* certain AI approaches work and others fail.

Max Kanat-Alexander's prescription for this systems-level thinking is deceptively simple: invest in the fundamentals. "Standardize environments. Improve deterministic validation. Refactor for testability and ability to reason about code base. Write down external context and intentions." He pointed out that agents "did not attend your verbal meeting that has no transcript"—a pointed reminder that tribal knowledge, the bane of human onboarding, is equally toxic to AI integration. Better documentation doesn't just help agents; it helps every developer who comes after you.

The testing insight particularly resonated. "We spend more time reading code than writing it, and even more so now," Kanat-Alexander observed. "Every software developer becomes a code reviewer." In an AI-assisted world, code review velocity becomes the bottleneck—which means organizations need faster, more deterministic validation processes. Tests that were "nice to have" for human developers become essential infrastructure for maintaining trust in agent-generated code.



Kath Korevec from Google Labs extended this trust framework into the future with her vision for proactive agents. Current tools are "fundamentally reactive," she argued, requiring developers to context-switch constantly to supervise AI work. The next generation—exemplified by Google's Jules autonomous coding agent—will build trust through "observation, timely action, personalization, and seamless integration." Trust comes from agents that know when to step in and what's missing, behaving like a skilled collaborator

rather than an eager but unpredictable assistant.

What emerges across these enterprise perspectives is a coherent theory of AI adoption: trust grows from predictable behavior, which requires solid infrastructure, which benefits humans and agents equally. The organizations succeeding aren't the ones deploying the fanciest models or building elaborate AI-specific workflows. They're the ones finally fixing their broken build processes, writing down their undocumented assumptions, and creating deterministic testing that works for everyone.

As Gene Kim's observation about trust suggests, this isn't fundamentally about AI capabilities at all. It's about creating systems where behavior can be predicted—and when prediction fails, understanding why. Support teams ship code not because they suddenly learned computer science, but because the system became predictable enough for motivated non-specialists to contribute safely. Managers commit code not because they're reclaiming hands-on work, but because the scaffolding around development became robust enough to support occasional contributors.

Max Kanat-Alexander's applause line deserves to be more than a memorable quote. "What's good for humans is good for AI" is a north star for enterprise AI strategy: stop building for the agents, and start building systems that work for everyone.

Theme 2: Context, Memory, and The Harness: Where Product Differentiation Really Happens

When OpenAI's Bill Chen and Brian Fioca took the stage at the AI Engineering Code Summit, they delivered a wake-up call: "Hard to track the models and we aren't making the problem easier for anybody." The ground is shifting so fast beneath our feet that betting on any single model is a recipe for obsolescence. Instead, they argued, the real competitive advantage lies in what they call "the harness"—the critical abstraction layer between models and production systems that encompasses prompts, agent loops, tools, and context management.

Gene's Hopes And Favorite Findings

DORA

- Observation: as I've worked with AI more, I trust the AI to do larger tasks
- One strange definition of trust: to what degree do can I predict how another party will act and react — the more I trust the other party, I can make bigger requests, with fewer words, with less feedback
- The notion of *fingerspitzengefühl*: person's instinctive ability to handle complex, uncertain situations with intuition, tact, and sensitivity (thank you Idan Gazit, GitHub)

"For some, the harness might be the special sauce of the product," Chen explained. At OpenAI, they've seen this play out across their Codex platform, which now processes dozens of trillions of tokens per week. The harness isn't just technical plumbing; it's the strategic surface area where steerability meets reliability, where intelligence combines with habit to create agents that remain useful across model versions.

The challenge is real. Custom tools can be out of distribution—models simply don't know how to use them without careful prompt engineering. Prompts themselves have poor portability across models. And latency issues force careful context management decisions. OpenAI's answer? Build tools like Codex Map that handle context management automatically, and design harnesses that drive steering better than prompt microtuning ever could. As Chen put it: "Steerability = intelligence + habit. Training has side effects."

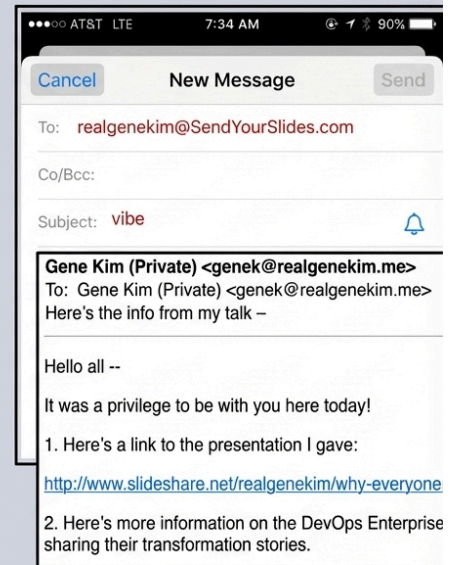
Want More Learn More?

To receive this presentation and the following:

- Excerpts of Vibe Coding and *Wiring the Winning Organization*
- Updates on benchmarking GenAI and developers
- The best talks from people on the frontier of using GenAI from the ETLIS Community
- PDF and audio excerpts from *The Unicorn Project*
- Eight excerpts from *Beyond The Phoenix Project* audio series w/John Willis
- The 140 page excerpt of *The DevOps Handbook*
- The 140 page excerpt of *The Phoenix Project*
- Videos and slides from DevOps Enterprise 2014-2019
- One hour excerpt of *The Phoenix Project* audiobook

Just pick up your phone, and send an email:

To: **realgenekim@SendYourSlides.com**
Subject: **vibe**



@RealGeneKim

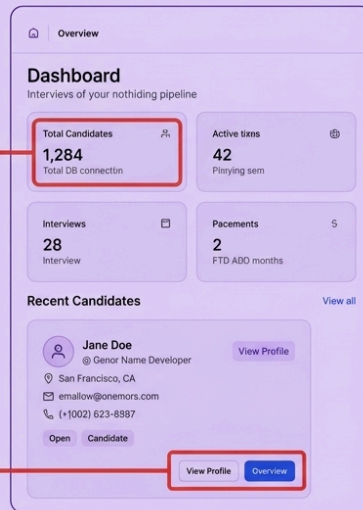
This convergence on context management wasn't limited to OpenAI. Across the summit, speakers returned again and again to the problem of keeping models focused and effective. At Replit, VP of AI Michele Catasta described how his team protects their main agent's working memory through an elegant architectural pattern: "Subagent invoked by the core loop with a task and fresh context." This approach prevents what Catasta calls "context pollution"—the gradual degradation of agent performance as irrelevant information accumulates in the context window.

For Replit's 22 million creators, many of whom are non-technical users, this architecture solves a critical UX problem. Catasta noted that more than 30% of features in user-generated apps are broken, and "users don't want to spend time doing testing." By giving subagents fresh context for each task, Replit prevents the "accumulation of small errors" and overcomes what Catasta wryly termed the "accumulations of whatevers" that plague frontier models. The core loop acts as orchestrator, deciding parallelism on the fly while maintaining a clean separation of concerns.

Testing fixes broken and hallucinated features

*Mock Data with
no DB connection*

*Button handlers
not hooked up*



Without testing, Agents
build “painted doors”

More than 30% of
individual features are
broken

**Almost every app has at
least one broken feature
or painted door**

At Anthropic, Head of API Engineering Katelyn Lesse offered a complementary perspective on context management. Her team has built what she calls "Memory + Context Editing"—and she's emphatic that this combination is "the way to go" for managing Claude's working memory. The Memory tool retrieves relevant context when needed, initially implemented through the file system. But the real innovation is Context Editing, which allows developers to manually clear unnecessary content, especially old tool results, from the context window.

CODE SUMMIT

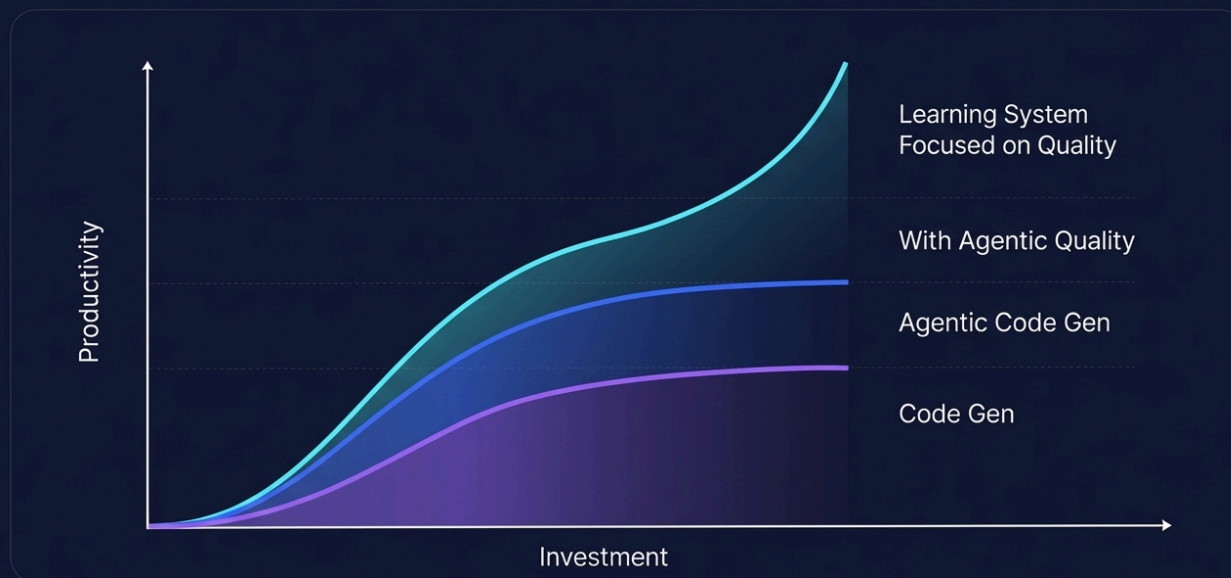
Keynote: The Future of AI in Code



Duane Moore
Jemaith and in Islett

"Effective agentic systems require both expanding capabilities and managing constraints," Lesse explained. It's not enough to give Claude more tools through MCP (Model Context Protocol) and code execution capabilities. You must also actively manage what stays in memory and what gets pruned. This dual focus—expansion and management—echoes the harness philosophy that Chen and Fioca articulated: the best systems don't just add features, they architect the entire interaction layer.

The importance of this architecture becomes clear when you consider the trust problem. Itamar Friedman from Qodo presented data showing that 76% of developers don't fully trust AI-generated code. When he asked why, the answer was revealing: "They don't trust the context that the LLM has." This isn't just about model capabilities—it's about whether the harness can maintain enough relevant context while filtering out noise. Qodo's response was to build their Context Engine, which pulls in logs, history, and PR comments to give models the right information at the right time.



The Browser Company's Samir Mody added another dimension to this conversation: treating model behavior as a craft. His team built prompt editors directly into their dev tools, moving all prompts into the browser itself. This 10x'd their iteration speed because engineers could ideate with full context. They even implemented GEPA (a prompt optimization technique) for hill-climbing refinement, turning what might have been a single-page prompt into a multipage system for generating skills based on user input.

What emerges from these conversations is a clear consensus: the harness is where the game is won or lost. Models will continue to improve—Chen and Fioca predict that "new models will raise the trust ceiling"—but the real differentiation happens in how you architect the layer between raw model capabilities and production reliability. Parallel tool execution, security sandboxing, context compaction, MCP support—these aren't ancillary features. They're the foundation on which autonomous agents are built.

As Chen advised: "Build where the models are going." That means investing in harnesses that can ride the wave of model improvements instead of drowning in constant rewrites. It means architecting for steerability through tools and context management, not just prompt engineering. And it means recognizing that the "special sauce" isn't in your model choice—it's in the abstraction layer you build around it.

Theme 3: SDLC Evolution: Good Software Engineering Practices, Amplified

The AI revolution in software development has revealed an uncomfortable truth: your codebase cleanliness is now your competitive advantage. Yegor Denisov-Blanch, a researcher with Stanford's Software Engineering Productivity Research Group, delivered findings from a landmark study of over 120,000 engineers that should make every engineering leader reconsider their technical debt backlog. "Clean code amplifies AI gains," Denisov-Blanch explained, presenting data that shows teams with high environment cleanliness scores saw productivity boosts of 35-40% from AI tools, while messy codebases saw gains of only 0-10%.

Redesign teams and operating models now - change takes time

Expect 3-5 models for different workflows - it's not one size fits all

Set a bold ambition - define specific targets to chase

Stay curious - learn from AI-native leaders

McKinsey & Company 21

This isn't a minor variation—it's a "rich gets richer effect" that threatens to bifurcate the software industry into those who can leverage AI and those who cannot. The Stanford research reveals that the quality of AI usage matters far more than quantity. "Token usage per model—not great predictive," Denisov-Blanch noted. What actually predicts productivity gains is the environment cleanliness index, a measure of how well-structured, documented, and maintainable a codebase is. The message is clear: invest in software cleanliness to unlock AI's potential, or watch your AI tools flounder in technical debt.

Max Kanat-Alexander, Executive Distinguished Engineer for Developer Experience at Capital One and author of "Code Simplicity," reinforced this thesis from the trenches of enterprise development. Drawing on 20+ years in the field and previous roles at Google and LinkedIn, Kanat-Alexander emphasized that fundamental software practices have become even more critical in the age of AI. "Better tooling, better testing, better messages—all that is old is new again, good software practices matter," he declared. His prescription for AI-ready organizations includes deterministic validation, refactoring for testability, standardized development environments, and—crucially—written documentation that captures external context and intentions.

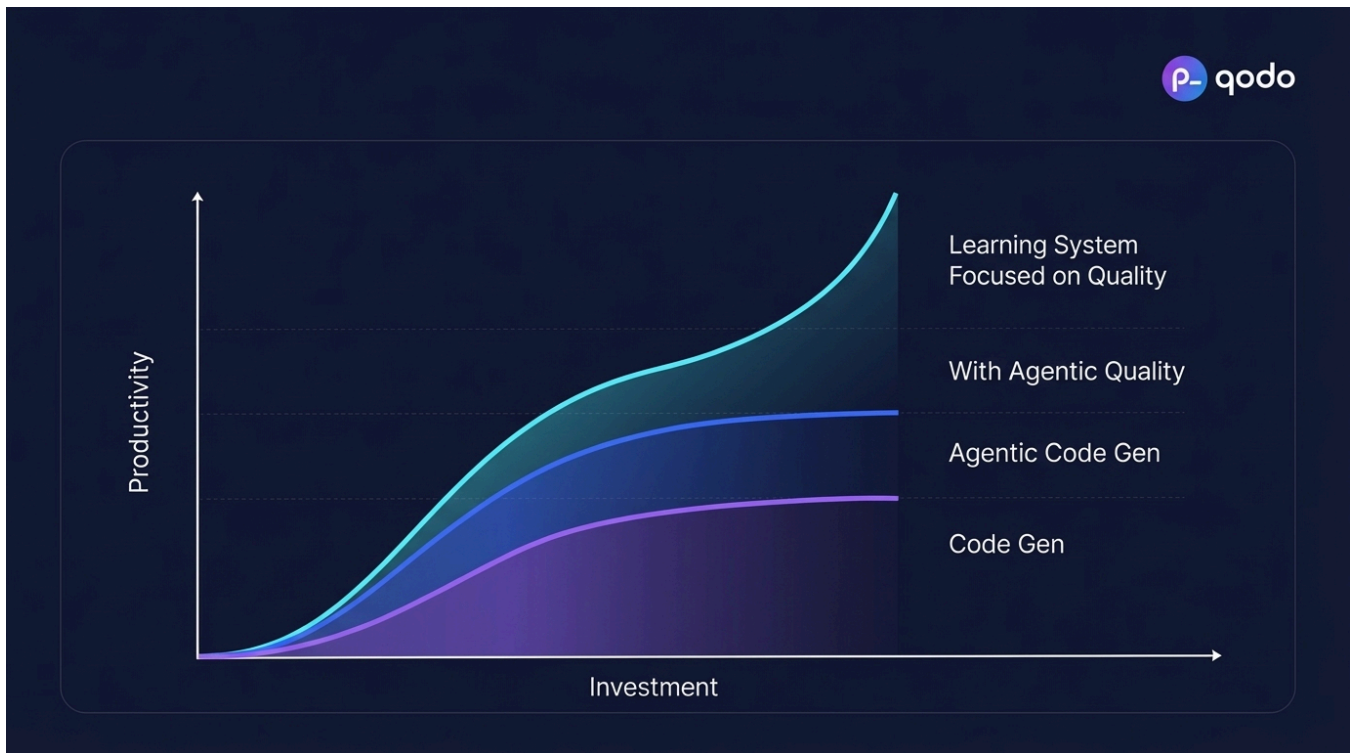
Code Review Quality

- **You have to encourage great reviews**
- **That hold a high bar and reject code that doesn't meet it**
- **Where junior engineers learn about coding *and* reviewing by getting reviews from seniors.**

"It did not attend your verbal meeting that has no transcript," Kanat-Alexander reminded the audience, highlighting how organizations relying on tribal knowledge are fundamentally incompatible with AI augmentation. His rallying cry—"What's good for humans is good for AI"—earned applause for articulating a truth many had suspected: investments in developer experience and code quality deliver returns regardless of AI's future trajectory.

But the amplification effect cuts both ways. Itamar Friedman, co-founder of Qodo, presented data from their 2025 report showing that while 82% of developers use AI assistants, 76% don't fully trust AI-generated code. The crisis, Friedman explained, isn't more bugs per PR—it's more PRs requiring review. "You have more bugs because there are more quantity of PRs, not because the PRs themselves are more buggy," he clarified. This volume problem

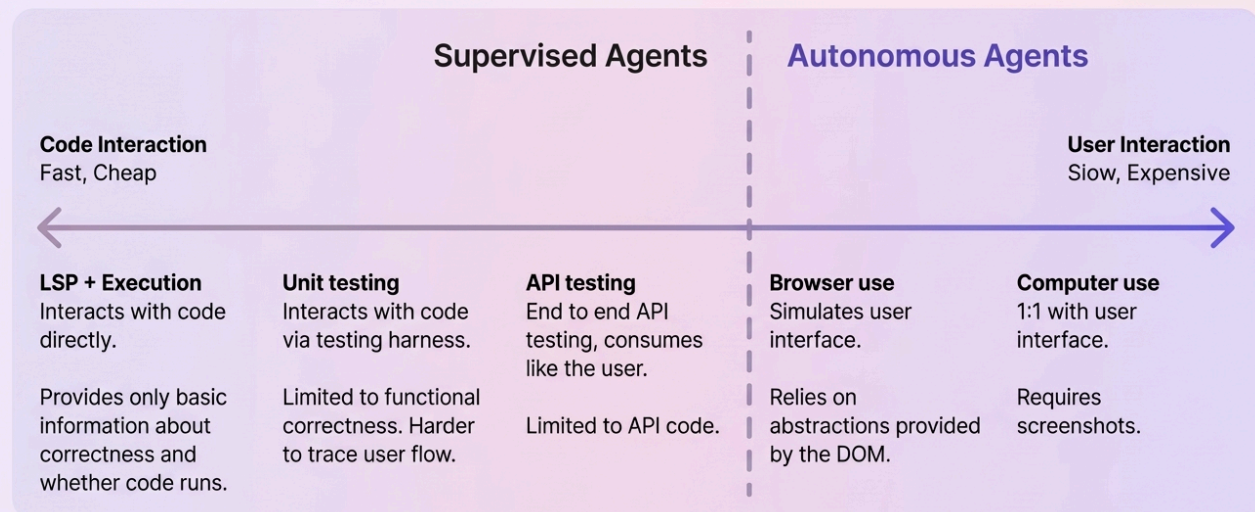
transforms code review from a bottleneck into a critical constraint that determines whether organizations can capitalize on AI productivity gains or drown in review backlogs.



The solution, according to Friedman, is autonomous testing and intelligent code review backed by rich context. Qodo's Context Engine pulls from logs, history, and PR comments to give AI reviewers the situational awareness they need to catch meaningful issues. "Don't accept this PR unless there is a minimum of testing," Friedman advocated, arguing that automated quality gates are essential guardrails. His data shows AI-assisted code review can deliver 2x productivity gains—but only when paired with rigorous standards.

Michele Catasta, VP of AI at Replit and former Head of Applied Research at Google Labs, offered a glimpse of how these principles play out in practice. Replit's autonomous agents build fully functional applications for 22 million creators, many of whom are non-technical. The key insight: autonomous testing breaks the feedback bottleneck and prevents "accumulations of whatevers"—the slow drift toward broken features that plagues iterative development. Catasta's agents dump memory to the filesystem, use the codebase itself for context, and invoke subagents with fresh context to avoid "context pollution." This architecture embodies Denisov-Blanch's cleanliness principle at the system level.

Spectrum of Code Verification

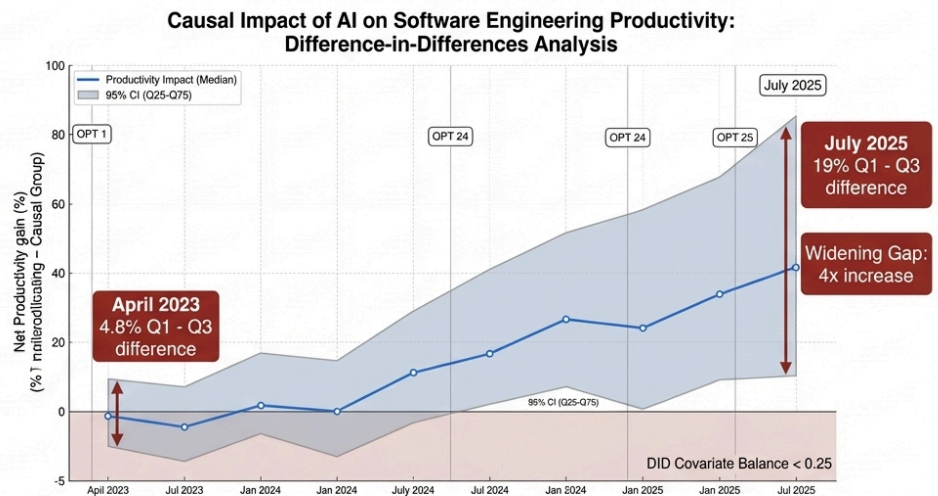


McKinsey's research, presented by Partner Martin Harrysson and Consultant Natasha Maniar, corroborates these findings from an organizational perspective. Their study of 120,000 developers found that successful AI adoption requires "getting a lot of small things right"—shorter sprints, smaller teams, increased investment in both greenfield and brownfield development. The bottleneck, they observed, is task allocation and change management. AI doesn't eliminate the need for good process; it makes good process more valuable by increasing throughput.

The convergence of these perspectives—from Stanford's academic research to Capital One's enterprise realities to Replit's consumer-facing products—points to a fundamental shift in how we should think about software development practices. AI isn't replacing the SDLC; it's acting as a force multiplier that rewards excellence and punishes mediocrity. Denisov-Blanch's environment cleanliness index, Kanat-Alexander's testability and documentation standards, Friedman's quality gates, and Catasta's context management strategies all describe different facets of the same underlying requirement: AI agents need structure to be effective.

Teams that master AI are accelerating their productivity gains, widening the gap with laggards

- 1 Identified 46 teams that used AI
- 2 Matched with 46 similar non-AI teams
- 3 Measured net productivity gains from AI quarterly



The Stanford research team modeled AI engineering practices across five levels: no AI use, opportunistic prompting, systematic prompting, agent-backed development, and orchestrated agentic workflows. But progression through these levels isn't automatic—it requires the foundational investments in code quality, testing infrastructure, documentation, and deterministic validation that the industry has sometimes treated as optional. In the age of AI, they're not. They're the difference between 35-40% productivity gains and 0-10%.

As Kanat-Alexander put it: "We spend more time reading code than writing it, and even more so now. Every software developer becomes a code reviewer." The practices that enable humans to review code efficiently—clear structure, comprehensive tests, explicit documentation—are precisely what enable AI to generate good code in the first place. The future of software development isn't about choosing between human craftsmanship and AI automation. It's about using AI to amplify the benefits of disciplined engineering—and accepting that without that discipline, AI offers little advantage at all.

Theme 4: The ROI Reality Check: Where AI Actually Delivers (And Where It Falls Flat)

The gap between demo and production has never felt wider. While vendors showcase AI coding assistants transforming greenfield projects in minutes, enterprises are wrestling with a far more stubborn truth: most real-world software development looks nothing like those polished demos. According to Stanford researcher Yegor Denisov-Blanch, who analyzed over 100,000 engineers across hundreds of companies, the numbers tell a story that should make every CTO pause before their next all-hands about AI-driven productivity.

Redesign teams and operating models now - change takes time

Expect 3-5 models for different workflows - it's not one size fits all

Set a bold ambition - define specific targets to chase

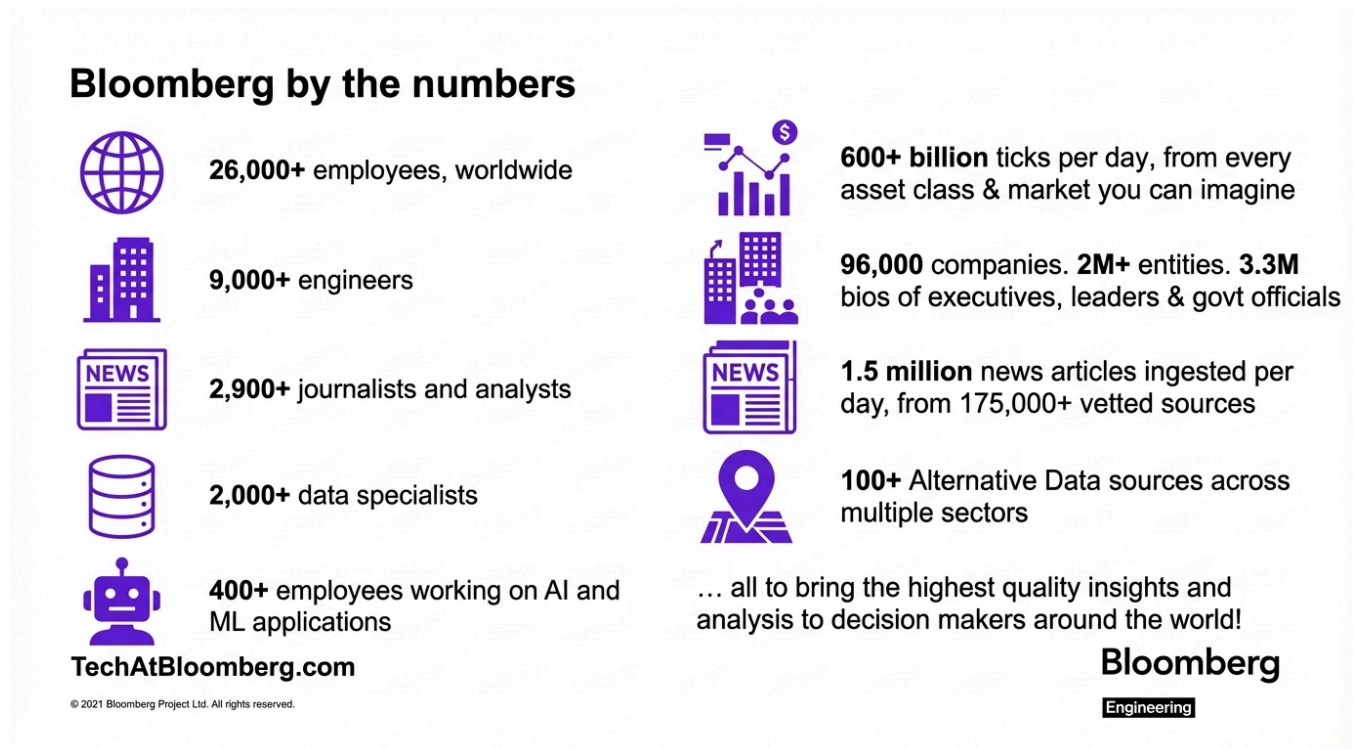
Stay curious - learn from AI-native leaders

McKinsey & Company 21

"AI usage quality means more than quantity," Denisov-Blanch emphasized, presenting findings that reveal a stark productivity divide. On greenfield projects and low-complexity tasks, AI coding tools deliver impressive 35-40% productivity gains. But throw those same tools at brownfield codebases and high-complexity work—the meat and potatoes of enterprise software development—and those gains plummet to 0-10%. The difference isn't just statistical noise; it's a fundamental reality check about where AI actually helps versus where it merely creates the illusion of progress.

This complexity gap manifests in unexpected ways. Bloomberg's Lei Zhang coined the term "vibe coding" to describe what happens when developers lean too heavily on AI assistants: "2

engineers can create the tech debt of 50 engineers." His team of 9,000+ engineers discovered that usage dropped precipitously when moving from greenfield to brownfield work—not because developers were resistant to AI, but because the tools simply couldn't navigate Bloomberg's tens of millions of lines of JavaScript and intricate internal libraries.



The data doesn't lie about AI's potential, but it demands honesty about its current limitations. NLW's survey of 3,500 use cases across eight impact categories revealed that coding is "further ahead" than other domains in delivering measurable ROI. Yet even in this leading category, the picture is more nuanced than headlines suggest. McKinsey's parallel study of 120,000 developers found that only 7% of organizations believe they're fully at scale with AI development tools. The remaining 93% are stuck in what multiple speakers called the "pilot purgatory"—endless experimentation without production-grade deployment.

Impact Categories



Time Savings
35.0% of use cases



Increased Output
14.5% of use cases



Quality Improvement
14.4% of use cases



New Capabilities
12.0% of use cases



Improved Decision Making
8.9% of use cases



Cost Savings
7.9% of use cases



Increased Revenue
4.2% of use cases



Risk Reduction
3.4% of use cases

Asaf Bord from Northwestern Mutual brought this reality into sharp focus when discussing his company's GenBI initiative at the Fortune 100 financial services giant. "The gap between demo and production is so broad," he stated plainly. Working in one of the world's most risk-averse environments—an organization built on 40-50 year client relationships—Bord identified four critical barriers: unknown technology, messy real data, blind-trust bias, and budget impact concerns.

Building Trust: Turning Bias-Land into Safe Ground

✓ What We Do	🔑 Why It Builds Trust
Sandbox Development PII-masked; Separate account, new models	Experiments isolated; No Prod impact
Crawl → Walk → Run Rollout Data-savvy SMEs → Managers → Execs	Early critics surface issues; later users inherit stability
Reuse Certified Content First Pre-cached queries & reports	Prevents hallucinations and “shadow” reports
Expectation Alignment Cut 80% lookup drudgery, not analyst jobs	Clear scope and pace = fewer surprises

Responsiveness

Confidence & Fluency

7

His solution? Stop working with synthetic data and start building trust incrementally. "Using actual data instead of synthetic to really understand the mess," Bord explained, Northwestern Mutual brought business stakeholders directly into the research project itself. Rather than building in isolation and hoping for adoption, they collected real questions users were asking their data systems—essentially creating organic evaluation sets from actual workflows. This approach of incremental ROI delivery, step by step, proved far more effective than big-bang transformations.

Can't We Just Ask ChatGPT to Query the Warehouse?



Schema dump is not scalable

Massive schemas overwhelm context windows, drive up cost, require constant sync.



Schema doesn't guarantee understanding

LLMs can't infer joins, decode shorthand, or align vague questions to exact fields.

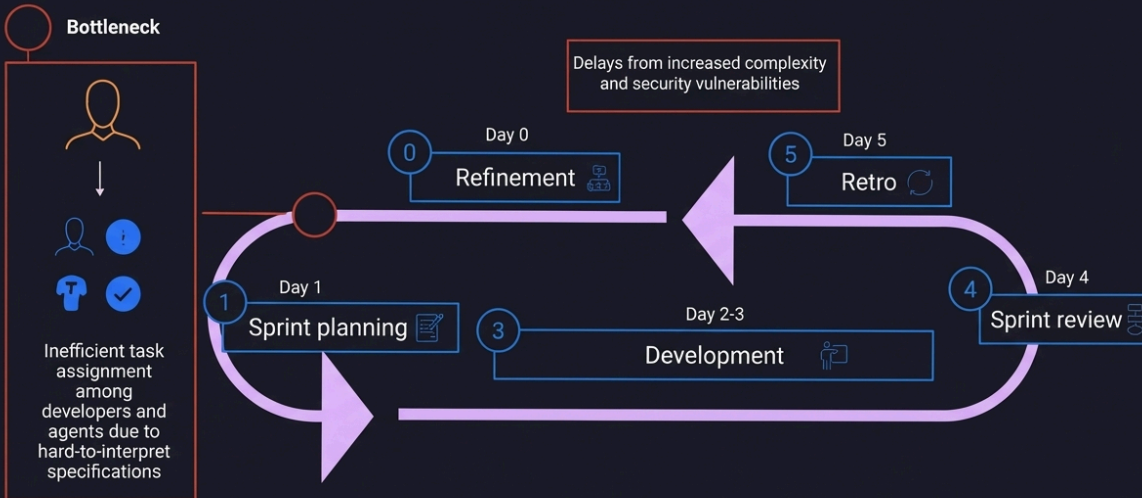


Governance is non-negotiable

Must enforce access rights, prefer certified reports, and use consistent taxonomy.

The trust-building imperative extends beyond individual projects to organizational change itself. McKinsey's research into post-Agile methodologies revealed that successful AI adoption requires "getting a lot of little things right"—numerous small interventions rather than wholesale process overhauls. Organizations are experimenting with shorter sprints, smaller teams, and increased investment in both greenfield and brownfield development, but the timeline for human change remains stubbornly long.

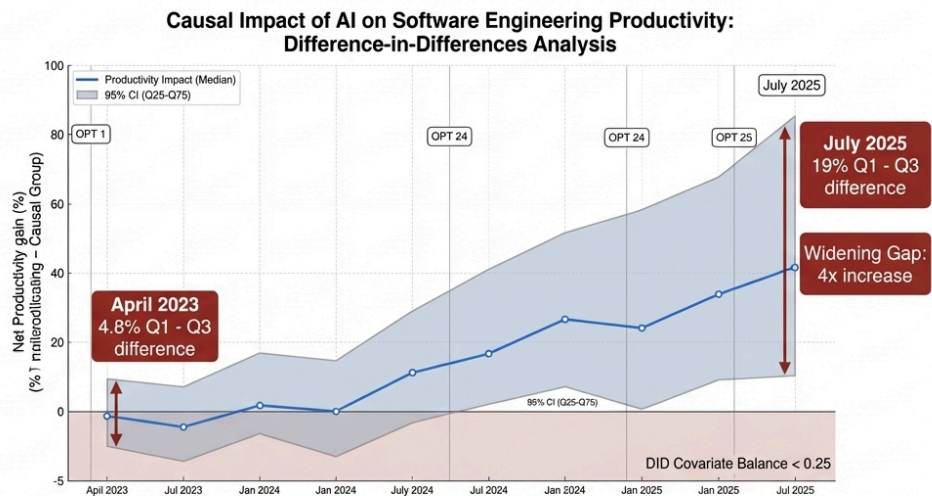
Bottlenecks within current operating model and team setup



Perhaps most counterintuitively, Denisov-Blanch's research identified "environment cleanliness" as a key factor amplifying AI productivity gains. Clean code doesn't just make human developers more effective—it dramatically improves AI tool performance. "Invest in software cleanliness to get the gains," he advised. "Fight the entropy." This finding flips conventional thinking: rather than using AI to compensate for messy codebases, organizations should clean up their code to unlock AI's potential.

Teams that master AI are accelerating their productivity gains, widening the gap with laggards

- 1 Identified 46 teams that used AI
- 2 Matched with 46 similar non-AI teams
- 3 Measured net productivity gains from AI quarterly



The research also revealed what Denisov-Blanch called a "rich gets richer effect" —the gap between top and bottom performers is widening, not shrinking, with AI adoption. Teams that already maintained clean codebases, established good practices, and invested in systematic tooling are seeing outsized benefits. Meanwhile, teams struggling with technical debt and process chaos find AI tools amplifying their existing problems rather than solving them.

Bloomberg's Zhang offered a reframing that cuts to the heart of the matter: AI "changes the cost function of engineering." Rather than asking what AI can do, his team asks what work developers don't want to do. This led to targeted applications like uplift agents for handling patches and rationale documentation, and incident response agents to parse overwhelming alert volumes—unglamorous but high-value use cases that deliver clear ROI.

The convergence of these studies points to an uncomfortable truth: AI coding tools aren't a silver bullet for productivity, they're an amplifier of existing organizational capabilities. Success requires working with actual messy data, bringing business stakeholders into the research process, maintaining code cleanliness, and building trust through incremental delivery. The 35-40% gains are real—but only for organizations willing to do the unglamorous work of getting their houses in order first.

As NLW's survey showed growing optimism about AI deployment over the course of 2024, the gap between pilot projects and production scale remains the defining challenge. The

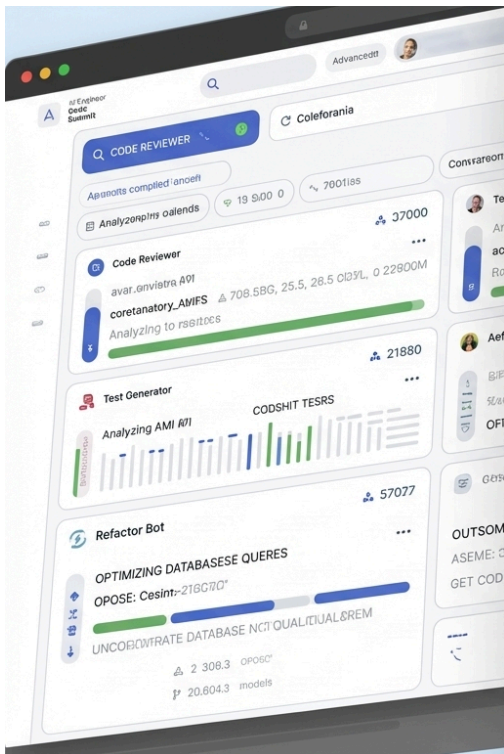
organizations that will thrive aren't those racing to adopt every new AI tool, but those methodically building the foundations—clean code, clear processes, stakeholder trust—that allow those tools to actually deliver value. The ROI is there, but it's earned through discipline, not demos.

Theme 5: Proactive vs Reactive: The UX Evolution

"Humans are unitaskers," declares Kath Korevec, Director of Product at Google Labs, cutting to the heart of why our current AI tools are failing us. We're building agents that wait to be asked, that sit idle in chat windows, that force us to context-switch and craft perfect prompts. But as Korevec frames it, we're stuck "reminding the husband to do the dishes"—managing our AI assistants rather than being supported by them. The fundamental question driving her work on Jules, Google's autonomous coding agent, isn't about making AI faster or smarter. It's about reducing the mental load entirely.

and suggested next steps.

Steve Yegge's provocative declaration that "if you are still using an IDE by Jan 1st you are a bad engineer" amplifies Korevec's vision from a different angle. Speaking at the same conference, Yegge argued for abandoning the "diver window" metaphor—where we give one agent a bigger context window (oxygen tank) and send it deep—in favor of ant-like swarms of agents working in parallel. His critique of current tools like Claude Code ("I use it 14 hours a day" but "ain't it") highlights the same problem Korevec identified: cognitive overhead. These are power tools that require constant supervision. "What damage can we do with untrained people? You can cut your foot off."



Adoption Depends on Having UI

We've established that CLI agents are too hard for most devs.

We clearly need a UI.

We also need to lift people up.

- Claude Code shows your nose in everything.
- You have to watch everything it does.
- We need helpers -- models!

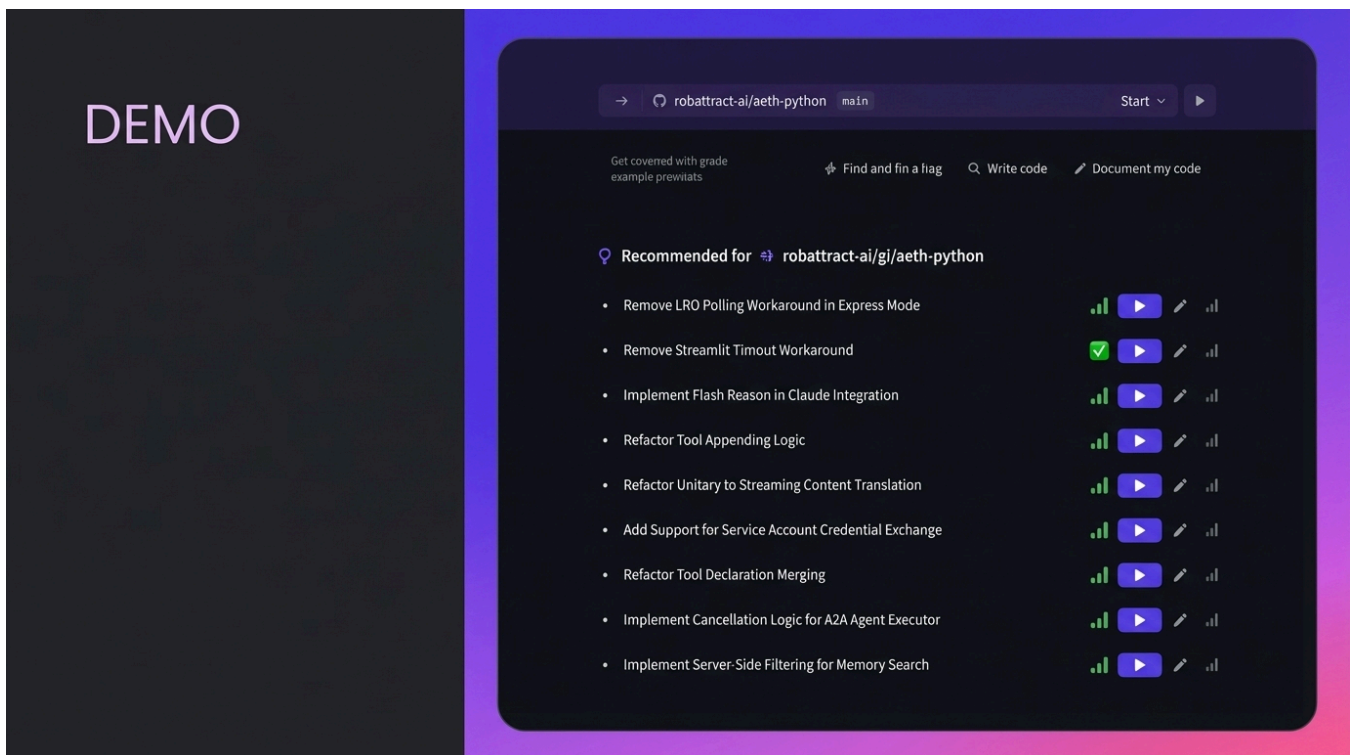
But coding agents aren't heading in this direction.

Instead, they're doubling down on the CLI.

- How much has Claude Code changed since February?

The answer, both speakers suggest, is moving supervision from the micro level (watching every action) to the macro level (setting goals and trusting systems). Michele Catasta from Replit frames this as maximizing "the irreducible runtime of the agent"—giving agents specific scope where they make all technical decisions while users maintain control over aspects they care about. The autonomy isn't about how long agents run, but about the completeness of the tasks they handle. Catasta's team discovered that more than 30% of features shipped by Replit Agent were broken because "users don't want to spend time doing testing." The solution? Autonomous testing that breaks the feedback bottleneck and prevents the "accumulation of whatevers" that frontier models tend to produce.

This evolution becomes tangible in Dan Shipper's operation at Every, where 15 people manage four software products generating seven-figure revenue. "AI permits engineers to work with fractured attention," Shipper observes—a statement that would horrify traditional engineering managers but captures the new reality. Engineers commit code to products they don't own. Managers commit code. New hires are productive on their first day because tacit knowledge has been codified into prompts. The agents aren't waiting for perfectly formed instructions; they're integrated into workflows that allow parallel development of multiple features and bugs simultaneously.



Korevec's warning resonates across all these implementations: "The patterns of how we use an IDE right now might not exist at all next year." She's not suggesting we abandon control, but rather that we "don't be afraid to question the new ways of building software." The proactive agent future isn't about AI doing everything—it's about AI removing the mental overhead of coordination, context management, and task sequencing that currently fragments developer attention.

The Browser Company's Samir Mody demonstrates what this looks like in practice with their prompt editor integrated directly into dev builds, achieving "10x the speed of ideating and iterating." But even as tools accelerate, Mody emphasizes that we're "in the early days of model behavior" as a craft. The shift from functional to agentic behavior requires new disciplines around behavior design, measurement, and model steering—treating how AI acts

as seriously as how it performs.

What emerges from these converging perspectives is less a death of traditional development than an evolution of partnership. Korevec's Jules doesn't replace the developer building the Halloween animatronic head—it amplifies creative agency by handling the tedious cycles. Yegge's ant swarms don't eliminate engineering judgment—they distribute execution. Shipper's fractured attention isn't chaos—it's enabled focus on what matters. The reactive chat interface represented AI waiting for us to figure out what to ask. Proactive agents represent AI figuring out what we need and suggesting it with rationale, learning from what we ignore, and reducing the overhead of staying in flow. As Korevec frames it, we're moving from tools that respond to tools that collaborate—and that shift changes everything about how software gets built.

Theme 6: The Economics of AI Engineering: New Cost Structures & Compensation Models

When AI can generate the same token output as a developer's annual salary for \$100 per day, the fundamental economics of software engineering collapse. This isn't a hypothetical scenario—it's the lived reality described by Gene Kim at the AI Engineering Code Summit, and it's forcing a radical rethinking of how we price products, compensate engineers, and calculate the true cost of development.

Arman Hezarkhani, Managing Partner at Tenex, is pioneering what may be the most radical response to this economic shift: paying engineers like salespeople. "We pay engineers based upon the story points they complete," Hezarkhani explains. "Paid on output. Uncapped upside. Incentivized to work smarter, faster, harder." At Tenex, engineers are compensated for completed work rather than hours logged, with multiple engineers on track to earn over \$1M annually—compensation levels traditionally reserved for top sales performers or executives.

CLIENT STORY



“THE RETAIL TECHNOLOGY COMPANY”

The Retail Technology Company's mission is to connect physical and digital stores through innovative store analytics and customer engagement solutions. The company has built an intelligent platform that collects, processes, and analyzes store data from various sources like video, sensors, and customer feedback, providing retailers with actionable insights into customer behavior, traffic patterns, and store operations.

THE COMPANY

We built their Store Intelligence (SaaS) platform that runs compact, high-accuracy AI at the edge, streams only encrypted data events to the cloud, and lights up existing dashboards with new metrics. Behind the scenes, we compressed large 'teacher' models into fast 'student' models for sensor-rich hardware, standardized and encrypted all data traffic, and exposed a single reporting API. The result is a secure, production-ready AI capability that helps retailers reduce costs, minimize abandonment, and optimize store layouts without ripping and replacing hardware.

THE WORK

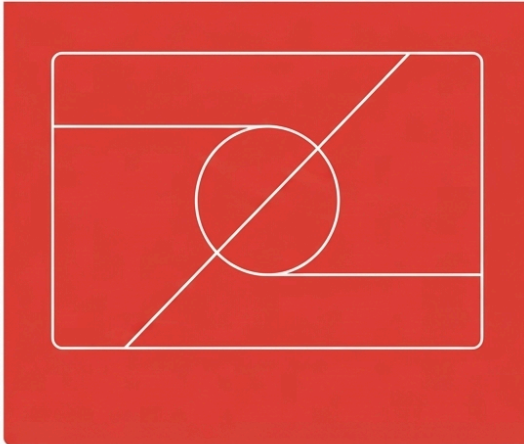
5%+

ANNUALIZED Y1 IN-STORE REVENUE UPLIFT

TENEX.CO IX

The logic is elegantly simple: if AI tools can 10x developer productivity, why should engineers be paid the same whether they embrace those tools or not? Traditional salary models create a perverse incentive structure where developers have no economic reason to adopt AI assistance. Output-based compensation flips this entirely. "Give your team a reason to go faster," Hezarkhani urges. When every story point completed translates directly to compensation, engineers become highly motivated to leverage every productivity enhancement available—especially AI coding assistants that can dramatically accelerate throughput.

The AI strategy playbook for senior executives



Tenex's model addresses common concerns through rigorous quality gates. To prevent story point inflation, strategy teams define scope upfront. To ensure quality doesn't suffer from speed, every deliverable undergoes three rounds of internal and external QA. And to avoid gaming the system, Tenex focuses on hiring engineers who understand that sustainable high performance, not corner-cutting, drives long-term earnings. The results speak for themselves: one billboard company client received a fully automated moderation system in two weeks; a retail technology client got on-device heat mapping models deployed in record time.

This compensation revolution reflects a deeper economic transformation. As Lei Zhang, Head of Technology Infrastructure at Bloomberg, notes: AI "**changes the cost function of engineering.**" When Bloomberg's 9,000+ engineers can suddenly accomplish tasks that previously required weeks of manual effort, the traditional cost-per-feature calculation becomes obsolete. But this also creates new challenges. Zhang observed that AI coding tool usage "dropped really quickly once we moved back from greenfield" projects, highlighting the disconnect between AI demos and messy production reality. His team's solution? Focus AI on tasks developers actively dislike—uplift agents for tedious migrations, incident response agents for alert overload—rather than trying to automate creative work.

Bloomberg by the numbers



26,000+ employees, worldwide



9,000+ engineers



2,900+ journalists and analysts



2,000+ data specialists



400+ employees working on AI and ML applications

TechAtBloomberg.com

© 2021 Bloomberg Project Ltd. All rights reserved.



600+ billion ticks per day, from every asset class & market you can imagine



96,000 companies. **2M+** entities. **3.3M** bios of executives, leaders & govt officials



1.5 million news articles ingested per day, from 175,000+ vetted sources



100+ Alternative Data sources across multiple sectors

... all to bring the highest quality insights and analysis to decision makers around the world!

Bloomberg

Engineering

The pricing implications extend beyond internal engineering costs to how software companies charge customers. Asaf Bord, GenAI Products Leader at Northwestern Mutual, directly addresses this shift: "How do we price software in this new era? Usage price vs seats price." The traditional seat-based model assumes fixed value per user, but AI's variable costs—token consumption, compute resources, API calls—create wildly different cost structures depending on actual usage. When a single complex query might consume exponentially more resources than a thousand simple ones, flat-rate pricing becomes economically untenable.

Building Trust: Turning Bias-Land into Safe Ground

✓ What We Do	🔑 Why It Builds Trust
Sandbox Development PII-masked; Separate account, new models	Experiments isolated; No Prod impact
Crawl → Walk → Run Rollout Data-savvy SMEs → Managers → Execs	Early critics surface issues; later users inherit stability
Reuse Certified Content First Pre-cached queries & reports	Prevents hallucinations and “shadow” reports
Expectation Alignment Cut 80% lookup drudgery, not analyst jobs	Clear scope and pace = fewer surprises

Responsiveness

Confidence & Fluency

7

Steve Yegge, who spends 14 hours daily using Claude Code despite claiming "Claude code ain't it," provides perhaps the starkest framing of the economic discontinuity. Cursor users versus Copilot users are "10x productive by any measure," he argues. This isn't incremental improvement—it's the difference between Swiss watchmakers and automated factories. When productivity gaps reach 10x, compensation models built on time-based salary simply can't capture the value creation differential.

Gene Kim drives the point home with his FAAFO framework (Faster, Ambitious, Alone, Fun, Options): when costs drop 100x, previously impossible projects become "annoying things that become free." Enterprise examples abound: [Booking.com](https://www.booking.com) elevating developer productivity across the organization, Travelopitia replacing legacy applications in 4-6 weeks with half the team size, Cisco requiring every senior developer to "vibe code" an application to production to build trust in the new paradigm.

The economics are undeniable. The question is whether traditional tech companies can adapt their compensation, pricing, and cost models fast enough—or whether startups like Tenex, unburdened by legacy structures, will capture the value creation this shift enables. As Hezarkhani's million-dollar engineers demonstrate, those who align incentives with the new economic reality won't just survive the transition—they'll thrive in it.

Conclusion: Building the Future on Solid Foundations

The AI Engineering Code Summit revealed a fundamental paradox: the path to AI-augmented software development runs directly through the fundamentals we've sometimes neglected. Clean code, comprehensive testing, clear documentation, predictable systems, psychological safety—these aren't obstacles to AI adoption, they're prerequisites.

The organizations succeeding share a common pattern: they're investing in foundations that benefit both humans and AI, building trust through incremental delivery, focusing on unglamorous but high-ROI use cases, and rethinking everything from compensation models to UX paradigms. The 35-40% productivity gains are real and achievable—but only for those willing to do the hard work of creating environments where AI agents can thrive.

As Max Kanat-Alexander's applause line captured: what's good for humans is good for AI. The future belongs to organizations that understand this insight and act on it systematically.